Licores Luna App

Jhon Deiby Perlaza Marin
Juan Pablo Ramirez Caracas
Juan Fernando Sanabria Aguilera
Jhoan David Tálaga Santander
Juan Pedro Valencia Camacho

Desarrollo de aplicaciones para dispositivos móviles

Luis Eduardo Ordoñez Palacios, Ph.D.
Escuela de Ingeniería de Sistemas y Computación

Universidad del Valle

Sede Palmira

Tecnología en Desarrollo de Software

2026

Descripción del proyecto

1.1 Nombre de la aplicación

Licores Luna

1.2 Descripción general y contexto

Licores Luna es una solución tecnológica enfocada en la gestión comercial e inventarial para el sector de venta minorista de licores. La aplicación busca centralizar las operaciones diarias en el punto de venta, permitiendo al administrador registrar transacciones, controlar las existencias físicas de mercancía en tiempo real, clasificar egresos monetarios y auditar el rendimiento económico mediante un dashboard financiero.

1.3 Planteamiento del problema

En el comercio minorista de licores, la gestión de inventario y el control del flujo de caja suelen realizarse mediante métodos informales o cuadernos físicos. Esta práctica genera inconsistencias recurrentes entre las existencias reales y las registradas, pérdida de la trazabilidad en las ventas realizadas en efectivo o transferencias electrónicas, e incapacidad para monitorear el dinero recaudado por concepto de depósitos de envases retornables.

Las consecuencias de esta problemática incluyen:

- Falta de visibilidad sobre la rentabilidad neta diaria del negocio.
- Pérdidas económicas por descuadres entre mercancía vendida y stock en bodega.
- Falta de visibilidad del stock actual, lo que lleva a desabastecimientos no planificados de productos de alta demanda.
- Complejidad al identificar los productos que aplican impuesto y al conciliar las compras de reabastecimiento a proveedores.

Por lo tanto, surge la necesidad de un sistema digitalizado, ágil y de rápida respuesta en dispositivos móviles que garantice la consistencia de los datos en cada transacción.

1.4 Público objetivo

Usuario principal: El administrador principal del establecimiento “Licores Luna”, enfocado en la operación diaria en punto de venta.

1.5 Justificación

El desarrollo de una aplicación móvil nativa en lenguaje Kotlin se justifica por las exigencias de rendimiento y fluidez operacional en un entorno de punto de venta. A diferencia de soluciones híbridas o webs, una aplicación nativa garantiza el uso eficiente de los recursos del hardware, una respuesta inmediata al interactuar con el catálogo de productos y una integración fluida con la plataforma de datos.

Adicionalmente, el proyecto adopta una arquitectura desacoplada utilizando Supabase como plataforma de Backend-as-a-Service (BaaS). Para minimizar el riesgo de inconsistencia en los datos y liberar de carga lógica al dispositivo móvil, las operaciones críticas de actualización de stock se gestionan mediante triggers automáticos en la base de datos.

1.6 Objetivos del proyecto

Desarrollar una aplicación móvil que permita la administración de inventario, el registro de transacciones de compra/venta y la generación de resúmenes financieros para el establecimiento “Licores Luna”.

Objetivos específicos

● Implementar un módulo de autenticación seguro para el administrador mediante Supabase Auth.
● Desarrollar el registro de ventas con cálculo automático de totales, depósito de envases y método de pago.
● Construir la gestión del catálogo e inventario, con actualización automática de stock mediante triggers en la base de datos.
● Registrar gastos operativos y de reabastecimiento para controlar las salidas de dinero.
● Diseñar un dashboard financiero con filtros por periodo que muestre ingresos, depósitos y gastos.

1.7 Alcance del proyecto

El alcance del proyecto abarca la fase de análisis, especificación de requerimientos, diseño, desarrollo y pruebas de la aplicación móvil nativa Licores Luna para dispositivos Android.

El alcance incluye: Autenticación y seguridad, Gestión de registro de ventas, Gestión de catálogo e inventario, Gestión de registro de gastos, Dashboard financiero.

No incluye: No contempla dentro del alcance integraciones con pasarelas de pago automáticas ni emisión automatizada de facturación electrónica.

2. Requerimientos del Sistema

En esta sección se presentan de forma resumida y declarativa los requerimientos funcionales y no funcionales que rigen el comportamiento técnico y operacional de la aplicación. Cada enunciado describe el comportamiento esperado del software; sin embargo, su especificación detallada en formato de Historias de Usuario(HU) , junto con sus respectivos detalles se encuentran formalizados y organizados dentro del Product Backlog del proyecto.

2.1 Requerimientos Funcionales (RF)

RF-01 — Autenticación y Control de Acceso

- **Descripción:** El sistema deberá permitir al administrador iniciar sesión de forma segura ingresando sus credenciales (correo electrónico y contraseña). El sistema

validará la autenticidad contra el servicio de autenticación de Supabase antes de dar acceso a las pantallas principales.

RF-02 — Módulo de Ingreso de Ventas (Punto de Venta)

- **Descripción:** El sistema deberá permitir al administrador seleccionar productos del catálogo, definir la cantidad deseada, seleccionar si aplica el cobro de depósito por envase retornable, elegir el método de pago (Efectivo o Transferencia) y registrar la transacción.

RF-03 — Administración del Catálogo de Productos (Creación, Consulta y Edición)

- **Descripción:** El sistema deberá ofrecer una vista unificada del catálogo que permita visualizar los productos registrados y proporcionar un formulario para crear nuevos productos o editar los existentes. Se podrá configurar la visibilidad pública del producto en el catálogo mediante un estado activo/inactivo.

RF-04 — Registro de Gastos Operativos

- **Descripción:** El sistema deberá permitir registrar las salidas de dinero destinadas a gastos de funcionamiento interno del negocio que no afecten el volumen de inventario.

RF-05 — Registro de Gastos por Reabastecimiento de Stock

- **Descripción:** El sistema deberá permitir registrar egresos monetarios destinados a la compra de mercancía a proveedores, asociando el ingreso de nuevas unidades de un producto específico al inventario.

RF-06 — Módulo de Resumen Financiero (Dashboard)

- **Descripción:** El sistema deberá presentar un panel analítico con los principales indicadores financieros del negocio, permitiendo aplicar filtros temporales.

2.2 Requerimientos No Funcionales (RNF)

RNF-01 — Plataforma y Entorno de Desarrollo

- **Descripción:** La aplicación móvil debe desarrollarse exclusivamente en forma nativa mediante el lenguaje Kotlin y la API SDK de Android, descartando el uso de marcos de trabajo híbridos o vistas encapsuladas en WebView.
- **Criterio:** Código 100% Kotlin con tiempo de renderizado de interfaz optimizado para la plataforma Android.

RNF-02 — Arquitectura de Servicios y Backend

- **Descripción:** El almacenamiento relacional, la autenticación y la gestión de la API se integrarán mediante el servicio en la nube Supabase.
- **Criterio:** Toda comunicación de la app debe ejecutarse mediante las librerías cliente oficiales de Supabase para Kotlin/Java.

RNF-03 — Lógica de Negocio e Integridad en Base de Datos

- **Descripción:** La actualización del inventario provocada por ventas (resta de stock) o reabastecimientos (suma de stock) debe ser procesada del lado del servidor utilizando *triggers* nativos en PostgreSQL/Supabase.
- **Criterio:** La aplicación móvil no realiza modificaciones numéricas directas sobre el atributo de stock en el cliente, únicamente invoca el registro de la transacción.

RNF-04 — Usabilidad y Eficiencia

- **Descripción:** La interfaz de usuario debe estar diseñada para operar en un entorno ágil de caja minorista, minimizando la cantidad de toques requeridos para registrar una venta.
- **Criterio:** La confirmación de una venta estándar debe completarse en no más de 4 pasos interactivos desde la pantalla principal del punto de venta.

2.3 Acceso al Product Backlog

Para consultar la estructuración completa de estos requerimientos se puede acceder mediante el siguiente link:

Licores Luna

3. Diseño de la Aplicación

El diseño de la interfaz y experiencia de usuario de Licores Luna se enfoca en la usabilidad y eficiencia para entornos de cobro y gestión rápida en punto de venta. La arquitectura visual sigue los lineamientos de Material Design para Android, implementando patrones de interfaz estándar como barra de navegación inferior, tarjetas informativas, botones de acción flotante y diálogos modales de confirmación para evitar errores transaccionales.

Para visualizar el mapa de navegación, la arquitectura de pantallas y la experiencia interactiva completa de la aplicación Licores Luna, se puede acceder al proyecto en Figma por siguiente enlace: Figma
LInk del: Mapa navegable

4. Gestión del Proyecto

4.1 Repositorio Git

Para el control de versiones y el trabajo colaborativo se ha dispuesto la creación de un repositorio en GitHub.

- **Estrategia de Ramas (Branching Strategy):**
  - **main:** Contiene el código en estado de producción estable.
  - **develop:** Branch de integración para las nuevas funcionalidades desarrolladas.
  - **feature/*:** Ramas secundarias por cada tarea o requerimiento en desarrollo (ej. feature/login-supabase, feature/ventas-ui).

- **Convención de Commits:** Commits descriptivos estructurados mediante el formato: tipo(módulo): descripción corta (ej. feat(catalogo): agregar casillas de impuesto y retornable).

Acceso al Repositorio: https://github.com/JhonPerlaza0614/LicoresLunaAPP2

4.2 Gestión de Tareas con Trello

Para la administración y trazabilidad del proyecto se ha seleccionado la herramienta **Trello**. Las tareas se organizan en las siguientes listas de control:

1. **Backlog:** Lista general de requerimientos funcionales, historias de usuario y tareas técnicas por iniciar.
2. **En Proceso (In Progress):** Actividades asignadas que se están ejecutando en el ciclo actual.
3. **En Revisión / QA:** Tareas finalizadas en código que están pasando por pruebas de interfaz y verificación con Supabase.
4. **Hecho (Done):** Requerimientos validados y fusionados en la rama principal.

Acceso al trello: Trello